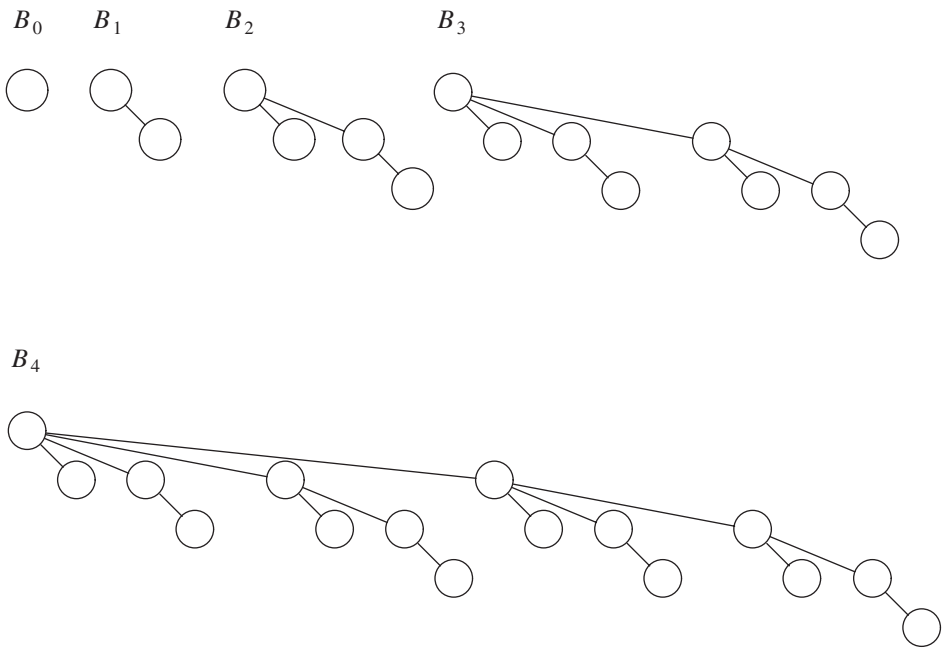

Figure 6.34 Binomial trees B_0, B_1, B_2, B_3, and B_4

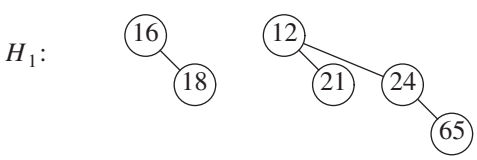

Figure 6.35 Binomial queue H_1 with six elements

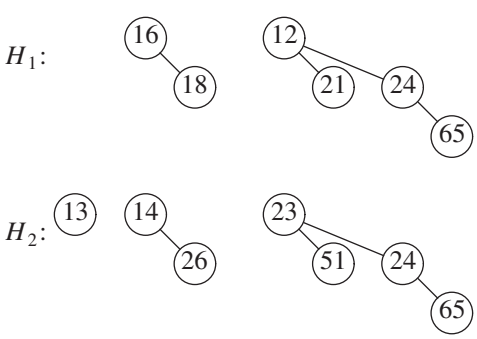

Figure 6.36 Two binomial queues H_1 and H_2

The merge is performed by essentially adding the two queues together. Let H_3 be the new binomial queue. Since H_1 has no binomial tree of height 0 and H_2 does, we can just use the binomial tree of height 0 in H_2 as part of H_3. Next, we add binomial trees of height 1. Since both H_1 and H_2 have binomial trees of height 1, we merge them by making the larger root a subtree of the smaller, creating a binomial tree of height 2, shown in Figure 6.37. Thus, H_3 will not have a binomial tree of height 1. There are now three binomial trees of height 2, namely, the original trees of H_1 and H_2 plus the tree formed by the previous step. We keep one binomial tree of height 2 in H_3 and merge the other two, creating a binomial tree of height 3. Since H_1 and H_2 have no trees of height 3, this tree becomes part of H_3 and we are finished. The resulting binomial queue is shown in Figure 6.38.

Since merging two binomial trees takes constant time with almost any reasonable implementation, and there are $O(\log N)$ binomial trees, the merge takes $O(\log N)$ time in the worst case. To make this operation efficient, we need to keep the trees in the binomial queue sorted by height, which is certainly a simple thing to do.

Insertion is just a special case of merging, since we merely create a one-node tree and perform a merge. The worst-case time of this operation is likewise $O(\log N)$. More precisely, if the priority queue into which the element is being inserted has the property that the smallest nonexistent binomial tree is B_i, the running time is proportional to $i + 1$. For example, H_3 (Fig. 6.38) is missing a binomial tree of height 1, so the insertion will terminate in two steps. Since each tree in a binomial queue is present with probability $\frac{1}{2}$, it follows that we expect an insertion to terminate in two steps, so the average time is constant. Furthermore, an analysis will show that performing N `inserts` on an initially empty binomial queue will take $O(N)$ worst-case time. Indeed, it is possible to do this operation using only $N - 1$ comparisons; we leave this as an exercise.

As an example, we show in Figures 6.39 through 6.45 the binomial queues that are formed by inserting 1 through 7 in order. Inserting 4 shows off a bad case. We merge 4

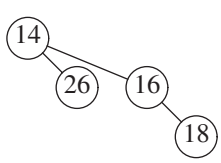

Figure 6.37 Merge of the two B_1 trees in H_1 and H_2

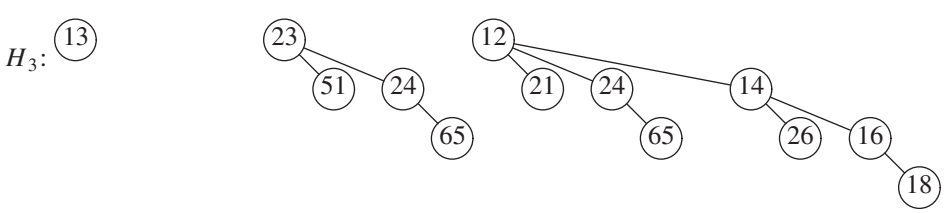

Figure 6.38 Binomial queue H_3: the result of merging H_1 and H_2

Figure 6.39 After 1 is inserted

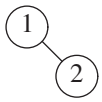

Figure 6.40 After 2 is inserted

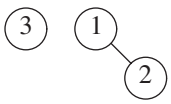

Figure 6.41 After 3 is inserted

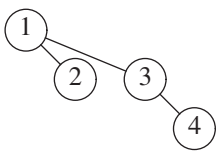

Figure 6.42 After 4 is inserted

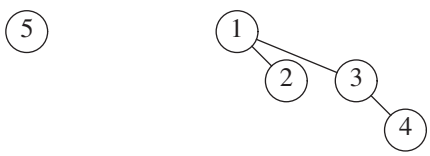

Figure 6.43 After 5 is inserted

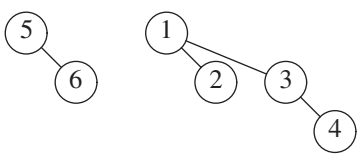

Figure 6.44 After 6 is inserted

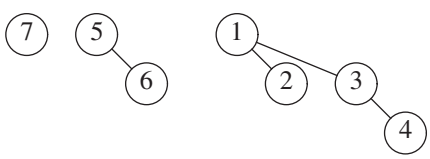

Figure 6.45 After 7 is inserted

with B_0, obtaining a new tree of height 1. We then merge this tree with B_1, obtaining a tree of height 2, which is the new priority queue. We count this as three steps (two tree merges plus the stopping case). The next insertion after 7 is inserted is another bad case and would require three tree merges.

A `deleteMin` can be performed by first finding the binomial tree with the smallest root. Let this tree be B_k, and let the original priority queue be H. We remove the binomial tree B_k from the forest of trees in H, forming the new binomial queue H'. We also remove the root of B_k, creating binomial trees $B_0, B_1, \ldots, B_{k-1}$, which collectively form priority queue H''. We finish the operation by merging H' and H''.

As an example, suppose we perform a `deleteMin` on H_3, which is shown again in Figure 6.46. The minimum root is 12, so we obtain the two priority queues H' and H'' in Figure 6.47 and Figure 6.48. The binomial queue that results from merging H' and H'' is the final answer and is shown in Figure 6.49.

For the analysis, note first that the `deleteMin` operation breaks the original binomial queue into two. It takes $O(\log N)$ time to find the tree containing the minimum element and to create the queues H' and H''. Merging these two queues takes $O(\log N)$ time, so the entire `deleteMin` operation takes $O(\log N)$ time.

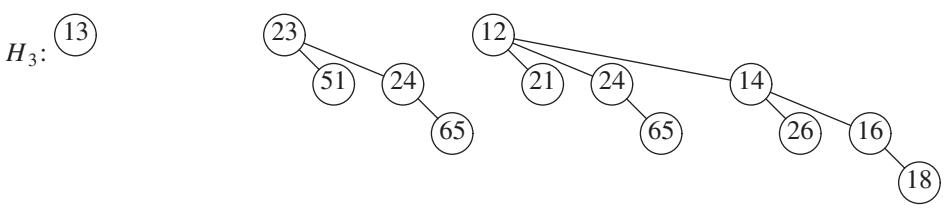

Figure 6.46 Binomial queue H_3

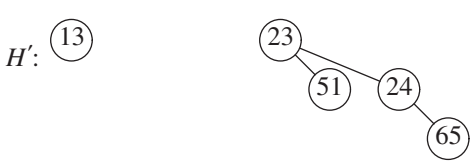

Figure 6.47 Binomial queue H', containing all the binomial trees in H_3 except B_3

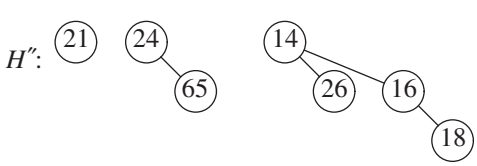

Figure 6.48 Binomial queue H'': B_3 with 12 removed

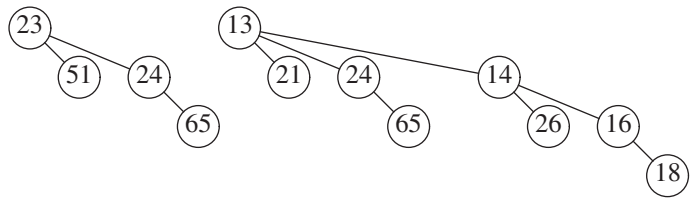

Figure 6.49 Result of applying `deleteMin` to H_3

6.8.3 Implementation of Binomial Queues

The `deleteMin` operation requires the ability to find all the subtrees of the root quickly, so the standard representation of general trees is required: The children of each node are kept in a linked list, and each node has a pointer to its first child (if any). This operation also requires that the children be ordered by the size of their subtrees. We also need to make sure that it is easy to merge two trees. When two trees are merged, one of the trees is added as a child to the other. Since this new tree will be the largest subtree, it makes sense to maintain the subtrees in decreasing sizes. Only then will we be able to merge two binomial trees, and thus two binomial queues, efficiently. The binomial queue will be an array of binomial trees.

To summarize, then, each node in a binomial tree will contain the data, first child, and right sibling. The children in a binomial tree are arranged in decreasing rank.

Figure 6.51 shows how the binomial queue in Figure 6.50 is represented. Figure 6.52 shows the type declarations for a node in the binomial tree and the binomial queue class interface.

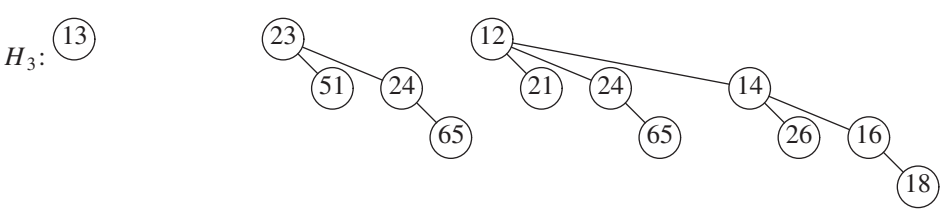

Figure 6.50 Binomial queue H_3 drawn as a forest

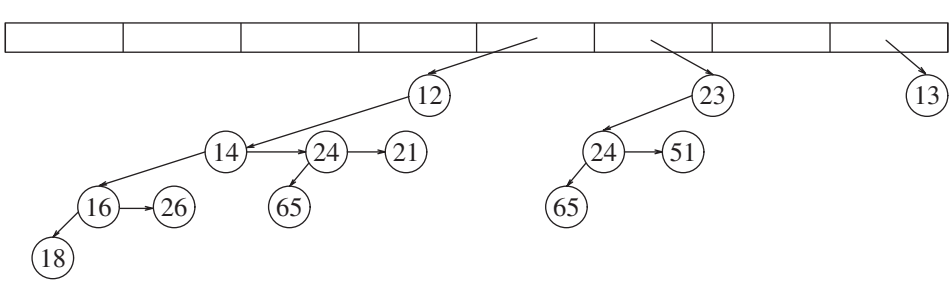

Figure 6.51 Representation of binomial queue H_3

```
template <typename Comparable>
class BinomialQueue
{
  public:
    BinomialQueue( );
    BinomialQueue( const Comparable & item );
    BinomialQueue( const BinomialQueue & rhs );
    BinomialQueue( BinomialQueue && rhs );

    ~BinomialQueue( );

    BinomialQueue & operator=( const BinomialQueue & rhs );
    BinomialQueue & operator=( BinomialQueue && rhs );

    bool isEmpty( ) const;
    const Comparable & findMin( ) const;

    void insert( const Comparable & x );
    void insert( Comparable && x );
    void deleteMin( );
    void deleteMin( Comparable & minItem );

    void makeEmpty( );
    void merge( BinomialQueue & rhs );

  private:
    struct BinomialNode
    {
        Comparable    element;
        BinomialNode *leftChild;
        BinomialNode *nextSibling;

        BinomialNode( const Comparable & e, BinomialNode *lt, BinomialNode *rt )
          : element{ e }, leftChild{ lt }, nextSibling{ rt } { }

        BinomialNode( Comparable && e, BinomialNode *lt, BinomialNode *rt )
          : element{ std::move( e ) }, leftChild{ lt }, nextSibling{ rt } { }
    };

    const static int DEFAULT_TREES = 1;

    vector<BinomialNode *> theTrees;  // An array of tree roots
    int currentSize;                  // Number of items in the priority queue

    int findMinIndex( ) const;
    int capacity( ) const;
    BinomialNode * combineTrees( BinomialNode *t1, BinomialNode *t2 );
    void makeEmpty( BinomialNode * & t );
    BinomialNode * clone( BinomialNode * t ) const;
};
```

Figure 6.52 Binomial queue class interface and node definition

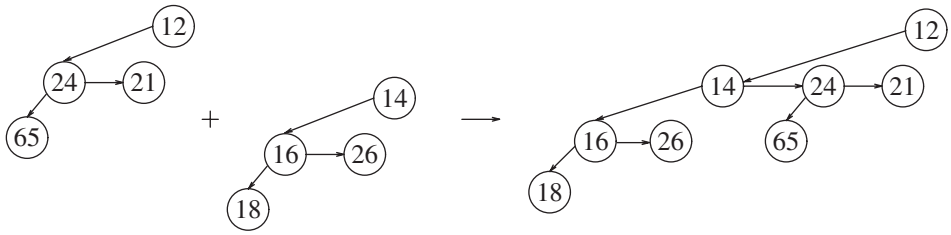

Figure 6.53 Merging two binomial trees

In order to merge two binomial queues, we need a routine to merge two binomial trees of the same size. Figure 6.53 shows how the links change when two binomial trees are merged. The code to do this is simple and is shown in Figure 6.54.

We provide a simple implementation of the `merge` routine. H_1 is represented by the current object and H_2 is represented by `rhs`. The routine combines H_1 and H_2, placing the result in H_1 and making H_2 empty. At any point we are dealing with trees of rank i. `t1` and `t2` are the trees in H_1 and H_2, respectively, and `carry` is the tree carried from a previous step (it might be `nullptr`). Depending on each of the eight possible cases, the tree that results for rank i and the `carry` tree of rank $i + 1$ is formed. This process proceeds from rank 0 to the last rank in the resulting binomial queue. The code is shown in Figure 6.55. Improvements to the code are suggested in Exercise 6.35.

The `deleteMin` routine for binomial queues is given in Figure 6.56 (on pages 280–281).

We can extend binomial queues to support some of the nonstandard operations that binary heaps allow, such as `decreaseKey` and `remove`, when the position of the affected element is known. A `decreaseKey` is a `percolateUp`, which can be performed in $O(\log N)$ time if we add a data member to each node that stores a parent link. An arbitrary `remove` can be performed by a combination of `decreaseKey` and `deleteMin` in $O(\log N)$ time.

```
/**
 * Return the result of merging equal-sized t1 and t2.
 */
BinomialNode * combineTrees( BinomialNode *t1, BinomialNode *t2 )
{
    if( t2->element < t1->element )
        return combineTrees( t2, t1 );
    t2->nextSibling = t1->leftChild;
    t1->leftChild = t2;
    return t1;
}
```

Figure 6.54 Routine to merge two equal-sized binomial trees

```
/**
 * Merge rhs into the priority queue.
 * rhs becomes empty. rhs must be different from this.
 * Exercise 6.35 needed to make this operation more efficient.
 */
void merge( BinomialQueue & rhs )
{
    if( this == &rhs )    // Avoid aliasing problems
        return;

    currentSize += rhs.currentSize;

    if( currentSize > capacity( ) )
    {
        int oldNumTrees = theTrees.size( );
        int newNumTrees = max( theTrees.size( ), rhs.theTrees.size( ) ) + 1;
        theTrees.resize( newNumTrees );
        for( int i = oldNumTrees; i < newNumTrees; ++i )
            theTrees[ i ] = nullptr;
    }

    BinomialNode *carry = nullptr;
    for( int i = 0, j = 1; j <= currentSize; ++i, j *= 2 )
    {
        BinomialNode *t1 = theTrees[ i ];
        BinomialNode *t2 = i < rhs.theTrees.size( ) ? rhs.theTrees[ i ]
                                                    : nullptr;
        int whichCase = t1 == nullptr ? 0 : 1;
        whichCase += t2 == nullptr ? 0 : 2;
        whichCase += carry == nullptr ? 0 : 4;

        switch( whichCase )
        {
          case 0: /* No trees */
          case 1: /* Only this */
            break;
          case 2: /* Only rhs */
            theTrees[ i ] = t2;
            rhs.theTrees[ i ] = nullptr;
            break;
          case 4: /* Only carry */
            theTrees[ i ] = carry;
            carry = nullptr;
            break;
```

Figure 6.55 Routine to merge two priority queues

```
                case 3: /* this and rhs */
                  carry = combineTrees( t1, t2 );
                  theTrees[ i ] = rhs.theTrees[ i ] = nullptr;
                  break;
                case 5: /* this and carry */
                  carry = combineTrees( t1, carry );
                  theTrees[ i ] = nullptr;
                  break;
                case 6: /* rhs and carry */
                  carry = combineTrees( t2, carry );
                  rhs.theTrees[ i ] = nullptr;
                  break;
                case 7: /* All three */
                  theTrees[ i ] = carry;
                  carry = combineTrees( t1, t2 );
                  rhs.theTrees[ i ] = nullptr;
                  break;
              }
          }

          for( auto & root : rhs.theTrees )
              root = nullptr;
          rhs.currentSize = 0;
      }
```

Figure 6.55 *(continued)*

```
      /**
       * Remove the minimum item and place it in minItem.
       * Throws UnderflowException if empty.
       */
      void deleteMin( Comparable & minItem )
      {
          if( isEmpty( ) )
              throw UnderflowException{ };

          int minIndex = findMinIndex( );
          minItem = theTrees[ minIndex ]->element;

```

Figure 6.56 `deleteMin` for binomial queues

```
        BinomialNode *oldRoot = theTrees[ minIndex ];
        BinomialNode *deletedTree = oldRoot->leftChild;
        delete oldRoot;

        // Construct H''
        BinomialQueue deletedQueue;
        deletedQueue.theTrees.resize( minIndex + 1 );
        deletedQueue.currentSize = ( 1 << minIndex ) - 1;
        for( int j = minIndex - 1; j >= 0; --j )
        {
            deletedQueue.theTrees[ j ] = deletedTree;
            deletedTree = deletedTree->nextSibling;
            deletedQueue.theTrees[ j ]->nextSibling = nullptr;
        }

        // Construct H'
        theTrees[ minIndex ] = nullptr;
        currentSize -= deletedQueue.currentSize + 1;

        merge( deletedQueue );
    }

    /**
     * Find index of tree containing the smallest item in the priority queue.
     * The priority queue must not be empty.
     * Return the index of tree containing the smallest item.
     */
    int findMinIndex( ) const
    {
        int i;
        int minIndex;

        for( i = 0; theTrees[ i ] == nullptr; ++i )
            ;

        for( minIndex = i; i < theTrees.size( ); ++i )
            if( theTrees[ i ] != nullptr &&
                theTrees[ i ]->element < theTrees[ minIndex ]->element )
                minIndex = i;

        return minIndex;
    }
```

Figure 6.56 *(continued)*

```
#include <iostream>
#include <vector>
#include <queue>
#include <functional>
#include <string>
using namespace std;

// Empty the priority queue and print its contents.
template <typename PriorityQueue>
void dumpContents( const string & msg, PriorityQueue & pq )
{
    cout << msg << ":" << endl;
    while( !pq.empty( ) )
    {
        cout << pq.top( ) << endl;
        pq.pop( );
    }
}

// Do some inserts and removes (done in dumpContents).
int main( )
{
    priority_queue<int>                          maxPQ;
    priority_queue<int,vector<int>,greater<int>>  minPQ;

    minPQ.push( 4 ); minPQ.push( 3 ); minPQ.push( 5 );
    maxPQ.push( 4 ); maxPQ.push( 3 ); maxPQ.push( 5 );

    dumpContents( "minPQ", minPQ );     // 3 4 5
    dumpContents( "maxPQ", maxPQ );     // 5 4 3

    return 0;
}
```

Figure 6.57 Routine that demonstrates the STL `priority_queue`; the comment shows the expected order of output

6.9 Priority Queues in the Standard Library

The binary heap is implemented in the STL by the class template named `priority_queue` found in the standard header file `queue`. The STL implements a max-heap rather than a min-heap so the largest rather than smallest item is the one that is accessed. The key member functions are:

```
void push( const Object & x );
const Object & top( ) const;
void pop( );
bool empty( );
void clear( );
```

`push` adds `x` to the priority queue, `top` returns the largest element in the priority queue, and `pop` removes the largest element from the priority queue. Duplicates are allowed; if there are several largest elements, only one of them is removed.

The priority queue template is instantiated with an item type, the container type (almost always you would want to use a `vector` that stores the items), and the comparator; defaults are allowed for the last two parameters, and the defaults yield a max-heap. Using a `greater` function object as the comparator yields a min-heap.

Figure 6.57 shows a test program that illustrates how the `priority_queue` class template can be used as both the default max-heap and a min-heap.

Summary

In this chapter, we have seen various implementations and uses of the priority queue ADT. The standard binary heap implementation is elegant because of its simplicity and speed. It requires no links and only a constant amount of extra space, yet supports the priority queue operations efficiently.

We considered the additional `merge` operation and developed three implementations, each of which is unique in its own way. The leftist heap is a wonderful example of the power of recursion. The skew heap represents a remarkable data structure because of the lack of balance criteria. Its analysis, which we will perform in Chapter 11, is interesting in its own right. The binomial queue shows how a simple idea can be used to achieve a good time bound.

We have also seen several uses of priority queues, ranging from operating systems scheduling to simulation. We will see their use again in Chapters 7, 9, and 10.

Exercises

6.1 Can both `insert` and `findMin` be implemented in constant time?

6.2 a. Show the result of inserting 10, 12, 1, 14, 6, 5, 8, 15, 3, 9, 7, 4, 11, 13, and 2, one at a time, into an initially empty binary heap.

b. Show the result of using the linear-time algorithm to build a binary heap using the same input.

6.3 Show the result of performing three `deleteMin` operations in the heap of the previous exercise.

6.4 A complete binary tree of N elements uses array positions 1 to N. Suppose we try to use an array representation of a binary tree that is not complete. Determine how large the array must be for the following:

a. a binary tree that has two extra levels (that is, it is very slightly unbalanced)
b. a binary tree that has a deepest node at depth $2 \log N$
c. a binary tree that has a deepest node at depth $4.1 \log N$
d. the worst-case binary tree

6.5 Rewrite the `BinaryHeap insert` routine by placing a copy of the inserted item in position 0.

6.6 How many nodes are in the large heap in Figure 6.13?

6.7 a. Prove that for binary heaps, `buildHeap` does at most $2N-2$ comparisons between elements.
b. Show that a heap of eight elements can be constructed in eight comparisons between heap elements.
** c. Give an algorithm to build a binary heap in $\frac{13}{8}N + O(\log N)$ element comparisons.

6.8 Show the following regarding the maximum item in the heap:
a. It must be at one of the leaves.
b. There are exactly $\lceil N/2 \rceil$ leaves.
c. Every leaf must be examined to find it.

** **6.9** Show that the expected depth of the kth smallest element in a large complete heap (you may assume $N = 2^k - 1$) is bounded by $\log k$.

6.10 * a. Give an algorithm to find all nodes less than some value, X, in a binary heap. Your algorithm should run in $O(K)$, where K is the number of nodes output.
b. Does your algorithm extend to any of the other heap structures discussed in this chapter?
* c. Give an algorithm that finds an arbitrary item X in a binary heap using at most roughly $3N/4$ comparisons.

** **6.11** Propose an algorithm to insert M nodes into a binary heap on N elements in $O(M + \log N \log\log N)$ time. Prove your time bound.

6.12 Write a program to take N elements and do the following:
a. Insert them into a heap one by one.
b. Build a heap in linear time.
Compare the running time of both algorithms for sorted, reverse-ordered, and random inputs.

6.13 Each `deleteMin` operation uses $2 \log N$ comparisons in the worst case.
* a. Propose a scheme so that the `deleteMin` operation uses only $\log N + \log\log N + O(1)$ comparisons between elements. This need not imply less data movement.
** b. Extend your scheme in part (a) so that only $\log N + \log\log\log N + O(1)$ comparisons are performed.
** c. How far can you take this idea?
d. Do the savings in comparisons compensate for the increased complexity of your algorithm?

6.14 If a d-heap is stored as an array, for an entry located in position i, where are the parents and children?

6.15 Suppose we need to perform M `percolateUps` and N `deleteMins` on a d-heap that initially has N elements.

a. What is the total running time of all operations in terms of M, N, and d?

b. If $d = 2$, what is the running time of all heap operations?

c. If $d = \Theta(N)$, what is the total running time?

★ d. What choice of d minimizes the total running time?

6.16 Suppose that binary heaps are represented using explicit links. Give a simple algorithm to find the tree node that is at implicit position i.

6.17 Suppose that binary heaps are represented using explicit links. Consider the problem of merging binary heap `lhs` with `rhs`. Assume both heaps are perfect binary trees, containing $2^l - 1$ and $2^r - 1$ nodes, respectively.

a. Give an $O(\log N)$ algorithm to merge the two heaps if $l = r$.

b. Give an $O(\log N)$ algorithm to merge the two heaps if $|l - r| = 1$.

★ c. Give an $O(\log^2 N)$ algorithm to merge the two heaps regardless of l and r.

6.18 A *min-max heap* is a data structure that supports both `deleteMin` and `deleteMax` in $O(\log N)$ per operation. The structure is identical to a binary heap, but the heap-order property is that for any node, X, at even depth, the element stored at X is smaller than the parent but larger than the grandparent (where this makes sense), and for any node, X, at odd depth, the element stored at X is larger than the parent but smaller than the grandparent. See Figure 6.58.

a. How do we find the minimum and maximum elements?

★ b. Give an algorithm to insert a new node into the min-max heap.

★ c. Give an algorithm to perform `deleteMin` and `deleteMax`.

★ d. Can you build a min-max heap in linear time?

★★ e. Suppose we would like to support `deleteMin`, `deleteMax`, and `merge`. Propose a data structure to support all operations in $O(\log N)$ time.

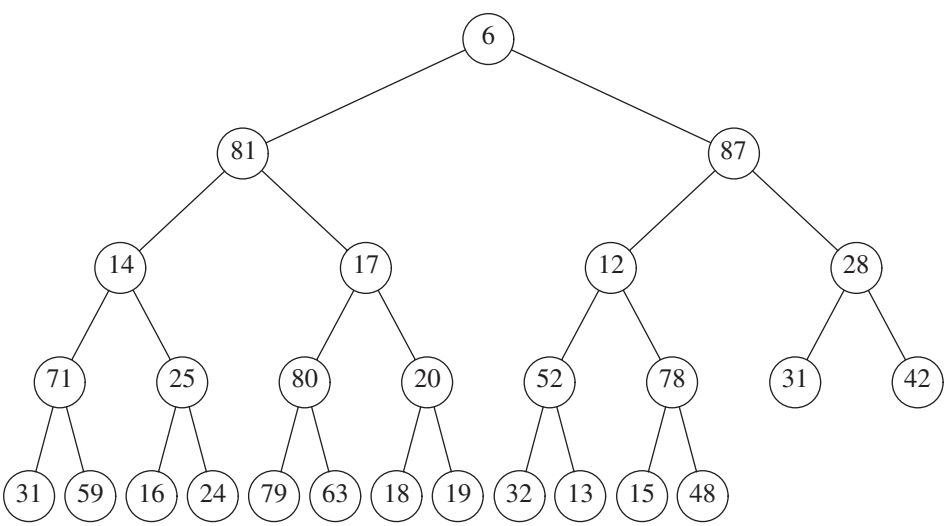

Figure 6.58 Min-max heap

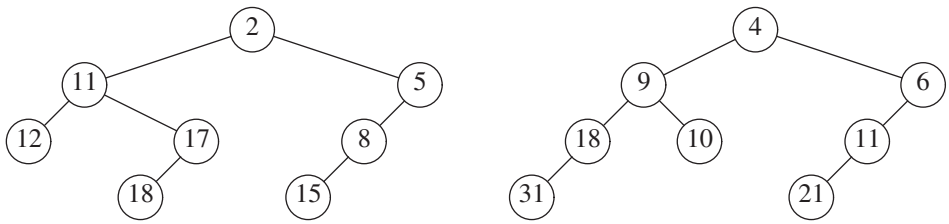

Figure 6.59 Input for Exercises 6.19 and 6.26

6.19 Merge the two leftist heaps in Figure 6.59.

6.20 Show the result of inserting keys 1 to 15 in order into an initially empty leftist heap.

6.21 Prove or disprove: A perfectly balanced tree forms if keys 1 to $2^k - 1$ are inserted in order into an initially empty leftist heap.

6.22 Give an example of input that generates the best leftist heap.

6.23 a. Can leftist heaps efficiently support `decreaseKey`?
b. What changes, if any (if possible), are required to do this?

6.24 One way to delete nodes from a known position in a leftist heap is to use a lazy strategy. To delete a node, merely mark it deleted. When a `findMin` or `deleteMin` is performed, there is a potential problem if the root is marked deleted, since then the node has to be actually deleted and the real minimum needs to be found, which may involve deleting other marked nodes. In this strategy, `remove`s cost one unit, but the cost of a `deleteMin` or `findMin` depends on the number of nodes that are marked deleted. Suppose that after a `deleteMin` or `findMin` there are k fewer marked nodes than before the operation.
* a. Show how to perform the `deleteMin` in $O(k \log N)$ time.
** b. Propose an implementation, with an analysis to show that the time to perform the `deleteMin` is $O(k \log(2N/k))$.

6.25 We can perform `buildHeap` in linear time for leftist heaps by considering each element as a one-node leftist heap, placing all these heaps on a queue, and performing the following step: Until only one heap is on the queue, dequeue two heaps, merge them, and enqueue the result.
a. Prove that this algorithm is $O(N)$ in the worst case.
b. Why might this algorithm be preferable to the algorithm described in the text?

6.26 Merge the two skew heaps in Figure 6.59.

6.27 Show the result of inserting keys 1 to 15 in order into a skew heap.

6.28 Prove or disprove: A perfectly balanced tree forms if the keys 1 to $2^k - 1$ are inserted in order into an initially empty skew heap.

6.29 A skew heap of N elements can be built using the standard binary heap algorithm. Can we use the same merging strategy described in Exercise 6.25 for skew heaps to get an $O(N)$ running time?

6.30 Prove that a binomial tree, B_k, has binomial trees $B_0, B_1, \ldots, B_{k-1}$ as children of the root.

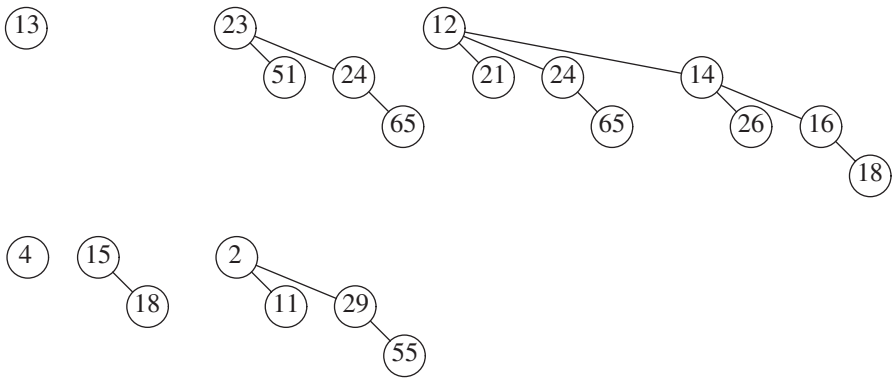

Figure 6.60 Input for Exercise 6.32

6.31 Prove that a binomial tree of height k has $\binom{k}{d}$ nodes at depth d.

6.32 Merge the two binomial queues in Figure 6.60.

6.33 a. Show that N `inserts` into an initially empty binomial queue take $O(N)$ time in the worst case.

b. Give an algorithm to build a binomial queue of N elements, using at most $N-1$ comparisons between elements.

* c. Propose an algorithm to insert M nodes into a binomial queue of N elements in $O(M + \log N)$ worst-case time. Prove your bound.

6.34 Write an efficient routine to perform `insert` using binomial queues. Do not call `merge`.

6.35 For the binomial queue

a. Modify the `merge` routine to terminate merging if there are no trees left in H_2 and the `carry` tree is `nullptr`.

b. Modify the `merge` so that the smaller tree is always merged into the larger.

** **6.36** Suppose we extend binomial queues to allow at most two trees of the same height per structure. Can we obtain $O(1)$ worst-case time for insertion while retaining $O(\log N)$ for the other operations?

6.37 Suppose you have a number of boxes, each of which can hold total weight C and items $i_1, i_2, i_3, \ldots, i_N$, which weigh $w_1, w_2, w_3, \ldots, w_N$, respectively. The object is to pack all the items without placing more weight in any box than its capacity and using as few boxes as possible. For instance, if $C = 5$, and the items have weights 2, 2, 3, 3, then we can solve the problem with two boxes.

In general, this problem is very hard, and no efficient solution is known. Write programs to implement efficiently the following approximation strategies:

* a. Place the weight in the first box for which it fits (creating a new box if there is no box with enough room). (This strategy and all that follow would give three boxes, which is suboptimal.)

b. Place the weight in the box with the most room for it.

* c. Place the weight in the most filled box that can accept it without overflowing.

** d. Are any of these strategies enhanced by presorting the items by weight?

6.38 Suppose we want to add the `decreaseAllKeys(Δ)` operation to the heap repertoire. The result of this operation is that all keys in the heap have their value decreased by an amount Δ. For the heap implementation of your choice, explain the necessary modifications so that all other operations retain their running times and `decreaseAllKeys` runs in $O(1)$.

6.39 Which of the two selection algorithms has the better time bound?

6.40 The standard copy constructor and `makeEmpty` for leftist heaps can fail because of too many recursive calls. Although this was true for binary search trees, it is more problematic for leftist heaps, because a leftist heap can be very deep, even while it has good worst-case performance for basic operations. Thus the copy constructor and `makeEmpty` need to be reimplemented to avoid deep recursion in leftist heaps. Do this as follows:

a. Reorder the recursive routines so that the recursive call to `t->left` follows the recursive call to `t->right`.
b. Rewrite the routines so that the last statement is a recursive call on the left subtree.
c. Eliminate the tail recursion.
d. These functions are still recursive. Give a precise bound on the depth of the remaining recursion.
*e. Explain how to rewrite the copy constructor and `makeEmpty` for skew heaps.

References

The binary heap was first described in [28]. The linear-time algorithm for its construction is from [14].

The first description of *d*-heaps was in [19]. Recent results suggest that 4-heaps may improve binary heaps in some circumstances [22]. Leftist heaps were invented by Crane [11] and described in Knuth [21]. Skew heaps were developed by Sleator and Tarjan [24]. Binomial queues were invented by Vuillemin [27]; Brown provided a detailed analysis and empirical study showing that they perform well in practice [4], if carefully implemented.

Exercise 6.7(b–c) is taken from [17]. Exercise 6.10(c) is from [6]. A method for constructing binary heaps that uses about $1.52N$ comparisons on average is described in [23]. Lazy deletion in leftist heaps (Exercise 6.24) is from [10]. A solution to Exercise 6.36 can be found in [8].

Min-max heaps (Exercise 6.18) were originally described in [1]. A more efficient implementation of the operations is given in [18] and [25]. Alternative representations for double-ended priority queues are the *deap* and *diamond deque*. Details can be found in [5], [7], and [9]. Solutions to 6.18(e) are given in [12] and [20].

A theoretically interesting priority queue representation is the *Fibonacci heap* [16], which we will describe in Chapter 11. The Fibonacci heap allows all operations to be performed in $O(1)$ amortized time, except for deletions, which are $O(\log N)$. *Relaxed heaps* [13] achieve identical bounds in the worst case (with the exception of `merge`). The procedure of [3] achieves optimal worst-case bounds for all operations. Another interesting implementation is the *pairing heap* [15], which is described in Chapter 12. Finally, priority queues that work when the data consist of small integers are described in [2] and [26].

1. M. D. Atkinson, J. R. Sack, N. Santoro, and T. Strothotte, "Min-Max Heaps and Generalized Priority Queues," *Communications of the ACM,* 29 (1986), 996–1000.
2. J. D. Bright, "Range Restricted Mergeable Priority Queues," *Information Processing Letters,* 47 (1993), 159–164.
3. G. S. Brodal, "Worst-Case Efficient Priority Queues," *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms* (1996), 52–58.
4. M. R. Brown, "Implementation and Analysis of Binomial Queue Algorithms," *SIAM Journal on Computing,* 7 (1978), 298–319.
5. S. Carlsson, "The Deap—A Double-Ended Heap to Implement Double-Ended Priority Queues," *Information Processing Letters,* 26 (1987), 33–36.
6. S. Carlsson and J. Chen, "The Complexity of Heaps," *Proceedings of the Third Symposium on Discrete Algorithms* (1992), 393–402.
7. S. Carlsson, J. Chen, and T. Strothotte, "A Note on the Construction of the Data Structure 'Deap'," *Information Processing Letters,* 31 (1989), 315–317.
8. S. Carlsson, J. I. Munro, and P. V. Poblete, "An Implicit Binomial Queue with Constant Insertion Time," *Proceedings of First Scandinavian Workshop on Algorithm Theory* (1988), 1–13.
9. S. C. Chang and M. W. Due, "Diamond Deque: A Simple Data Structure for Priority Deques," *Information Processing Letters,* 46 (1993), 231–237.
10. D. Cheriton and R. E. Tarjan, "Finding Minimum Spanning Trees," *SIAM Journal on Computing,* 5 (1976), 724–742.
11. C. A. Crane, "Linear Lists and Priority Queues as Balanced Binary Trees," *Technical Report STAN-CS-72-259,* Computer Science Department, Stanford University, Stanford, Calif., 1972.
12. Y. Ding and M. A. Weiss, "The Relaxed Min-Max Heap: A Mergeable Double-Ended Priority Queue," *Acta Informatica,* 30 (1993), 215–231.
13. J. R. Driscoll, H. N. Gabow, R. Shrairman, and R. E. Tarjan, "Relaxed Heaps: An Alternative to Fibonacci Heaps with Applications to Parallel Computation," *Communications of the ACM,* 31 (1988), 1343–1354.
14. R. W. Floyd, "Algorithm 245: Treesort 3," *Communications of the ACM,* 7 (1964), 701.
15. M. L. Fredman, R. Sedgewick, D. D. Sleator, and R. E. Tarjan, "The Pairing Heap: A New Form of Self-adjusting Heap," *Algorithmica,* 1 (1986), 111–129.
16. M. L. Fredman and R. E. Tarjan, "Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms," *Journal of the ACM,* 34 (1987), 596–615.
17. G. H. Gonnet and J. I. Munro, "Heaps on Heaps," *SIAM Journal on Computing,* 15 (1986), 964–971.
18. A. Hasham and J. R. Sack, "Bounds for Min-max Heaps," *BIT,* 27 (1987), 315–323.
19. D. B. Johnson, "Priority Queues with Update and Finding Minimum Spanning Trees," *Information Processing Letters,* 4 (1975), 53–57.
20. C. M. Khoong and H. W. Leong, "Double-Ended Binomial Queues," *Proceedings of the Fourth Annual International Symposium on Algorithms and Computation* (1993), 128–137.
21. D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching,* 2d ed., Addison-Wesley, Reading, Mass., 1998.
22. A. LaMarca and R. E. Ladner, "The Influence of Caches on the Performance of Sorting," *Proceedings of the Eighth Annual ACM-SIAM Symposium on Discrete Algorithms* (1997), 370–379.